# Critic-based PPO for Flow-Matching Models

Algorithm, Pseudocode, and Theoretical Analysis (Flow-Factory)

Flow-Factory

June 19, 2026

**Abstract**

This note documents the critic-based Proximal Policy Optimization (PPO) trainer in Flow-Factory (`trainer_type: ppo`). It is a *coupled* actor–critic algorithm: it reuses the same SDE rollout and per-step log-probability machinery as GRPO, but replaces GRPO's single, group-normalized advantage (broadcast to every denoising step) with a learned, timestep-conditioned *value critic* and Generalized Advantage Estimation (GAE), giving fine-grained per-step credit assignment. We formalize denoising as a finite-horizon MDP, give the data-flow diagram, pseudocode, and a short theoretical analysis (variance reduction, GAE bias–variance, the clipped surrogate). We close with a detailed comparison to *Explicit Critic Guidance for Aligning Diffusion Models* (arXiv:2605.27736), which makes the diffusion backbone its own value function—precisely the "backbone-critic" variant Flow-Factory evaluated and deferred.

## Contents

# 1 Introduction and Motivation

Flow-Factory fine-tunes flow-matching / diffusion models with online RL against (typically non-differentiable) reward models. The default algorithm, GRPO, treats a whole denoising rollout as a single "action": it computes one scalar advantage per sample from *group-relative* reward normalization and broadcasts it to all training timesteps. This is simple and strong, but has two structural limitations:

1. **No per-step credit.** Every denoising step in a trajectory shares the same advantage, so the gradient cannot distinguish which steps were responsible for a high/low reward.
2. **Group-mean baseline.** The variance-reduction baseline is the per-group reward mean; it ignores the *state* (the noisy latent), and its quality depends on the group size.

Critic-based PPO addresses both by learning a state-conditioned value function $V_\phi(\boldsymbol{z}_s, t_s)$ over noisy latent states and forming *per-step* advantages with GAE. The contribution of this implementation is a *lightweight, model-agnostic* actor–critic that drops into the existing rollout pipeline:

- an independent value critic that consumes the full generation latent at any layout (conv / packed / video) and is built eagerly from the channel count (Section 4.1);
- GAE per-step advantages with optional global whitening (Section 4.2);
- a PPO objective with policy-ratio clipping and value clipping, a critic warmup phase, and two optimizers sharing one backward (Section 4.3).

# 2 Preliminaries: Denoising as an MDP

**SDE sampling.** A rollout produces a trajectory of latents $\boldsymbol{z}_0 \to \boldsymbol{z}_1 \to \cdots \to \boldsymbol{z}_S$, where $\boldsymbol{z}_0$ is initial noise and $\boldsymbol{z}_S$ decodes to the generated image/video. At each stochastic (SDE) step $s$ the scheduler (Flow-SDE) defines a Gaussian transition (the *policy*)

$$\pi_\theta(\boldsymbol{z}_{s+1} \mid \boldsymbol{z}_s, t_s) = \mathcal{N}(\boldsymbol{z}_{s+1};\, \boldsymbol{\mu}_\theta(\boldsymbol{z}_s, t_s),\, \sigma_s^2 \boldsymbol{I}), \qquad \sigma_s = \texttt{std\_dev\_t} \cdot \sqrt{-\mathrm{d}t}, \tag{1}$$

whose mean $\boldsymbol{\mu}_\theta$ is a deterministic function of the network's velocity prediction. The per-step log-probability $\log \pi_\theta$ is tractable and is exactly what the scheduler returns and stores during rollout, which is the foundation of train–inference consistency.

**MDP.** We treat the ordered SDE steps as a finite-horizon MDP:

| | |
|---|---|
| state | $s_s = (\boldsymbol{z}_s, t_s)$ (noisy latent + scheduler timestep) |
| action | $a_s = \boldsymbol{z}_{s+1}$ (the sampled next latent) |
| policy | $\pi_\theta(a_s \mid s_s)$ from Eq. (1) |
| reward | $r_s = R \cdot \mathbb{1}[s = S{-}1]$, with terminal $R = \sum_k w_k\, r_k(\text{decode}(\boldsymbol{z}_S))$ |
| value | $V_\phi(\boldsymbol{z}_s, t_s)$, terminal $V(\boldsymbol{z}_S) \equiv 0$ |

The reward is *terminal only*: it is the (weighted) reward-model score of the final decoded sample, received on the last SDE transition. Figure 1 illustrates the episode.

# 3 From GRPO to Critic-based PPO

**GRPO.** For a group $g$ of $K$ rollouts sharing a prompt, GRPO assigns each sample the same advantage at *every* step:

$$A_i^{\text{GRPO}} = \frac{R_i - \text{mean}_{j \in g}(R_j)}{\text{std}(R)}, \qquad A_{i,s}^{\text{GRPO}} = A_i^{\text{GRPO}} \;\; \forall s. \tag{2}$$

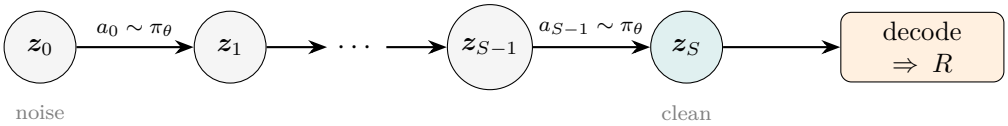


Figure 1: Denoising rollout as a finite-horizon MDP. The policy is the per-step SDE Gaussian transition; the reward is terminal (on the decoded final latent $\boldsymbol{z}_S$). The critic estimates $V_\phi(\boldsymbol{z}_s, t_s)$ at every step.

**PPO.** Critic-based PPO instead learns $V_\phi$ and forms per-step advantages by GAE (Section 4.2). The baseline is now state-aware ($V_\phi(\boldsymbol{z}_s, t_s)$ rather than a group mean), and credit is distributed per step. Table 1 summarizes the structural difference; everything else (rollout, per-step log-probabilities, SDE dynamics, optional KL-to-reference) is shared.

| | **GRPO** | **Critic-based PPO** |
|---|---|---|
| Advantage | one scalar / sample, broadcast | per-step $A_s$ via GAE |
| Baseline | per-group reward mean | learned $V_\phi(\boldsymbol{z}_s, t_s)$ |
| Extra parameters | none | lightweight value critic + its optimizer |
| Reward normalization | group-relative (mean/std) | none on $R$; advantages whitened |
| Per-epoch hook order | `sample()` → `prepare_feedback()` → `optimize()` | |

Table 1: GRPO vs. critic-based PPO. Only the credit-assignment / baseline differs.

## 4 Algorithm and Data Flow

Figure 2 shows the per-epoch data flow. The trainer reuses `generate_samples` for rollout, then computes *old* values with the current critic (no grad), runs GAE, stores per-step targets on each sample, and finally optimizes.

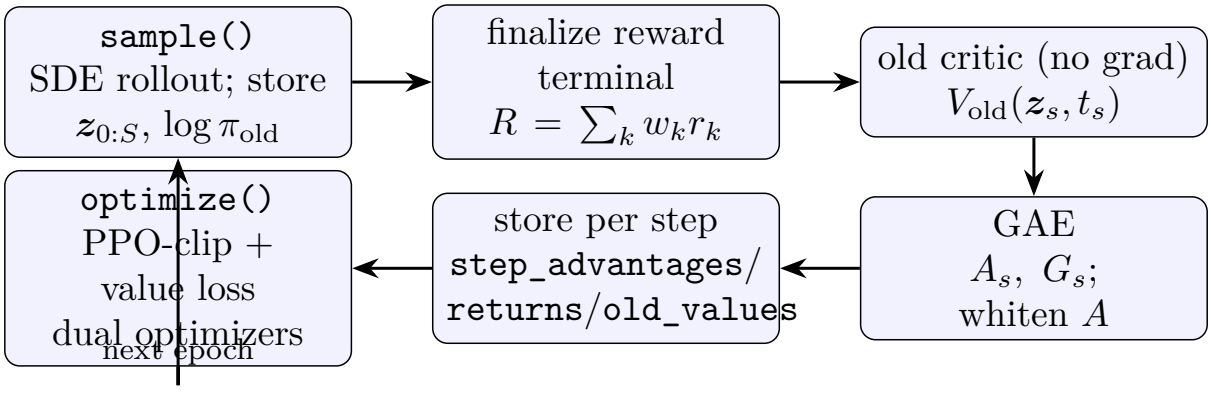


Figure 2: Per-epoch pipeline of the PPO trainer. `prepare_feedback()` covers the reward→old-value→GAE→store stages; `optimize()` does the gradient update.

### 4.1 Value critic

The critic is an independent, resolution-agnostic network (`trainers/ppo/critic.py`). Any latent layout is folded to $(B, \text{seq}, C)$ by moving the channel axis last (`latents_to_tokens` using `resolve_latent_axes`); tokens are encoded, aggregated over the sequence by *learned-query*

*multi-head attention pooling* (PMA-style, $O(\text{seq})$), concatenated with a sinusoidal timestep embedding, and mapped to a scalar (Figure 3). It is built eagerly in `_initialization()` from the channel count returned by `BaseAdapter.compute_actual_latent_shape`, so it needs no rollout to size and its checkpoint stays valid across resolutions.

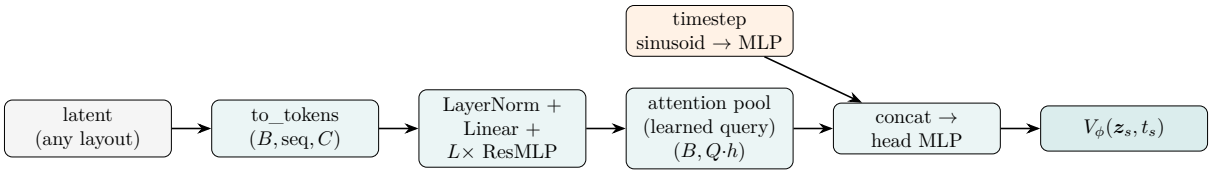


Figure 3: Value-critic architecture. Attention pooling keeps the network resolution/frame agnostic (it only depends on the channel count $C$) while preserving more information than mean/std pooling.

### 4.2 Generalized Advantage Estimation

With $V_{\text{old}}$ evaluated at each ordered SDE step, the TD residual and GAE advantage are

$$\delta_s = r_s + \gamma\, V(\boldsymbol{z}_{s+1}) - V(\boldsymbol{z}_s), \qquad V(\boldsymbol{z}_S) \equiv 0, \tag{3}$$

$$A_s = \sum_{l=0}^{S-1-s} (\gamma\lambda)^l\, \delta_{s+l}, \qquad G_s = A_s + V(\boldsymbol{z}_s), \tag{4}$$

computed by the backward recursion $A_s = \delta_s + \gamma\lambda A_{s+1}$ (`compute_gae`). $G_s$ is the value-regression target. With a terminal-only reward and $\gamma = \lambda = 1$ this collapses to $A_s = R - V(\boldsymbol{z}_s)$ and $G_s = R$ for all $s$ (the critic-baselined analogue of GRPO); $\lambda < 1$ trades variance for bias. The example config uses a *full, contiguous* SDE trajectory so the bootstrap $V(\boldsymbol{z}_{s+1})$ is the value of the actual next stored latent. Advantages are optionally whitened across all (sample, step) pairs and ranks via a single all-reduce (`whiten` → `utils.dist.global_tensor_stats`).

### 4.3 PPO objective

Let $\rho_s = \exp(\log \pi_\theta(a_s \mid \boldsymbol{z}_s) - \log \pi_{\text{old}}(a_s \mid \boldsymbol{z}_s))$ be the per-step importance ratio and $\hat{A}_s$ the (clamped, whitened) advantage. The clipped objective is

$$\mathcal{L}^{\text{pol}} = -\,\mathbb{E}_s\big[\min\left(\rho_s \hat{A}_s,\ \text{clip}(\rho_s, 1{+}c_-, 1{+}c_+)\,\hat{A}_s\right)\big], \tag{5}$$

$$\mathcal{L}^{\text{val}} = \tfrac{1}{2}\,\mathbb{E}_s\big[\max\left((V_\phi - G_s)^2,\ (V_\phi^{\text{clip}} - G_s)^2\right)\big], \quad V_\phi^{\text{clip}} = V_{\text{old}} + \text{clip}(V_\phi{-}V_{\text{old}}, \pm\epsilon_v), \tag{6}$$

$$\mathcal{L} = \mathcal{L}^{\text{pol}} + c_v\,\mathcal{L}^{\text{val}} \ +\ \beta\,\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}}). \tag{7}$$

As in Flow-GRPO, the ratio clip range $(c_-, c_+)$ is intentionally tiny (default $\pm 10^{-4}$) because $\log \pi$ is a per-dimension average over a high-dimensional Gaussian. The KL-to-reference term ($\beta =$ `kl_beta`) is optional and off by default. Policy and critic use *separate* backward passes (each only traverses its own sub-graph) and each steps its own AdamW optimizer (`optimizer`, `critic_optimizer`); this independence is what lets their update frequencies be decoupled (§4.5).

**Critic warmup as an up-front bootstrap.** The initial critic warmup and the periodic re-warmup (§4.6) share a single critic-only path. Before the epoch loop, BootstrapCritic resamples fresh rollouts and fits *only* the critic until it has taken $\sim$ `critic_warmup_steps` optimizer steps, so a value baseline is established before any policy update (the "value pretraining" idea of [1]). Unlike a warmup woven into the first epochs, this bootstrap runs once up front and does *not* freeze them: once the loop starts the policy updates every epoch.

### 4.4 Pseudocode

The algorithm has two committed stages; both are kept for reference. Algorithm 1 is the original formulation (commit `4bdec9d`): a single combined loss $\mathcal{L} = c_v\mathcal{L}^{\text{val}} + \mathcal{L}^{\text{pol}}(+\beta\,\text{KL})$, one `accelerator.backward`, a shared `accumulate` window, both optimizers stepping together at the sync boundary, and the warmup woven into the first epochs. Algorithm 2 is the current formulation (commit `4145161`): separate per-network backward passes, decoupled update intervals (§4.5), and the warmup unified with periodic critic re-warmup (§4.6).

**Algorithm 1** Critic-based PPO — v1: combined loss, shared `accumulate` (commit `4bdec9d`)

$\mathcal{D} \leftarrow$ SAMPLE(); $\{R_i\} \leftarrow$ REWARD($\mathcal{D}$); $\mathcal{S} \leftarrow$ sorted(train SDE steps)
$V_{\text{old}} \leftarrow V_\phi(\mathcal{D})$ (no grad); $A, G \leftarrow$ GAE; $A \leftarrow$ WHITEN($A$); store on samples
**for** inner epoch $= 1, \ldots, N$ **do**
  **for** micro-batch $b$ **do** ▷ `with accumulate(model_bundle, critic)`
    **for** $s \in \mathcal{S}$ **do**
      $V \leftarrow V_\phi(\boldsymbol{z}_s^b, t_s)$; $\mathcal{L} \leftarrow c_v\mathcal{L}^{\text{val}}$
      **if** $r \geq$ `critic_warmup_steps` **then**
        $\mathcal{L}^{\text{pol}} \leftarrow$ Eq. (5); $\mathcal{L} \leftarrow \mathcal{L} + \mathcal{L}^{\text{pol}}(+\,\beta\,\text{KL})$ ▷ policy forward
      **else**
        policy frozen; forward skipped
      **end if**
      BACKWARD($\mathcal{L}$) ▷ single combined backward; accelerate $\div gas$
      **if** `sync_gradients` at round end **then**
        clip + step `critic_optimizer` (and `optimizer` unless warmup); zero both
      **end if**
    **end for**
  **end for**
**end for**

**Algorithm 2** Critic-based PPO — current: separate backward, decoupled frequency + re-warmup (commit `4145161`)

**once, before training:** WarmupCritic(`critic_warmup_steps`) (§4.6); **every** `critic_warmup_interval` **rounds:** WarmupCritic(`critic_rewarmup_steps`)
$\mathcal{D} \leftarrow$ Sample() ▷ SDE rollout; store $\boldsymbol{z}_{0:S}$, $\log \pi_{\text{old}}$ at SDE steps
$\{R_i\} \leftarrow$ Reward($\mathcal{D}$) ▷ terminal weighted-sum score per sample
$\mathcal{S} \leftarrow$ sorted(train SDE steps)
**for** micro-batch $b$ in $\mathcal{D}$ **do** ▷ no grad
  **for** $s \in \mathcal{S}$ **do**
    $V_{\text{old}}[b, s] \leftarrow V_\phi(\boldsymbol{z}_s^b, t_s)$
  **end for**
**end for**
$A, G \leftarrow \text{GAE}(V_{\text{old}}, R, \gamma, \lambda)$; $A \leftarrow$ Whiten($A$) ▷ global, cross-rank
store $A[\cdot, s], G[\cdot, s], V_{\text{old}}[\cdot, s]$ on each sample
**for** inner epoch $= 1, \ldots, N$ **do**
  **for** micro-batch $b$ **do** ▷ round $r$; *gas* micro-steps/round; $c_i, p_i$ = update intervals
    **for** $s \in \mathcal{S}$ **do**
      **if** `update_critic_in_optimize` **then** ▷ mode A; mode B trains critic via bursts
        $V \leftarrow V_\phi(\boldsymbol{z}_s^b, t_s)$; Backward($c_v \mathcal{L}^{\text{val}} / c_i$) ▷ no_sync off only at critic boundary
      **end if**
      $\mathcal{L}^{\text{pol}} \leftarrow$ Eq. (5); Backward($(\mathcal{L}^{\text{pol}} + \beta\,\text{KL})/p_i$) ▷ policy every micro-step
      **if** round end and mode A and $c_i \mid (r{+}1)$ **then**
        clip + step `critic_optimizer`; zero grad
      **end if**
      **if** round end and $p_i \mid (r{+}1)$ **then**
        clip + step `optimizer`; zero grad
      **end if**
    **end for**
  **end for**
**end for**

**Algorithm 3** GAE (terminal-only reward)

**Require:** values $V \in \mathbb{R}^{B \times S}$, terminal rewards $R \in \mathbb{R}^B$, $\gamma, \lambda$
$A \leftarrow \mathbf{0}_{B \times S}$; $A_{\text{next}} \leftarrow \mathbf{0}_B$
**for** $s = S - 1, \ldots, 0$ **do**
  $r \leftarrow R$ if $s = S - 1$ else $\mathbf{0}$; $V_{\text{next}} \leftarrow \mathbf{0}$ if $s = S - 1$ else $V[:, s{+}1]$
  $\delta \leftarrow r + \gamma V_{\text{next}} - V[:, s]$
  $A_{\text{next}} \leftarrow \delta + \gamma\lambda A_{\text{next}}$; $A[:, s] \leftarrow A_{\text{next}}$
**end for**
**return** $A$, $A + V$

### 4.5 Decoupling critic and policy update frequency

Because the critic and policy use *separate* backward passes, their optimizer steps can run at different cadences. Let a *round* be one base gradient-accumulation window of *gas* micro-steps. Each network keeps its own window: with intervals $c_i =$ `critic_update_interval` and $p_i =$ `policy_update_interval` (in rounds), the critic accumulates $c_i\,gas$ micro-steps before each step and the policy accumulates $p_i\,gas$. **Both networks still forward and backward on every micro-step**, so no data is dropped; a larger interval only enlarges the effective batch

and lowers the step frequency of that network. Setting $c_i{=}1, p_i{=}K$ makes the critic relatively $K\times$ faster; swapping them reverses the direction.

**Gradient scaling.** `accelerator.backward` already divides the loss by $gas$. To make the accumulated gradient the *mean* over a network's full (interval $\cdot$ $gas$)-micro-step window, each loss is additionally divided by its own interval before backward:

$$g \;=\; \frac{1}{c_i\, gas}\sum_{j=1}^{c_i\, gas}\nabla_\phi(c_v\mathcal{L}_j^{\text{val}}), \qquad g \;=\; \frac{1}{p_i\, gas}\sum_{j=1}^{p_i\, gas}\nabla_\theta(\mathcal{L}_j^{\text{pol}}{+}\beta\,\mathrm{KL}_j), \tag{8}$$

so the effective learning rate is invariant to the interval. Gradient synchronization is deferred with per-network `no_sync` until each window closes; window boundaries are tracked by a self-maintained micro-step counter, not Accelerate's shared `sync_gradients`. Since the prepared optimizers' `step`/`zero_grad` are themselves gated on that flag, we pin `sync_gradients=True` at `optimize`/burst entry so they always fire, and (manual-`gas` only) assert the per-`optimize` micro-step count is a multiple of `gas` so a window never straddles two epochs' rollouts. This is the two-timescale idea of TD3 / DMD2 (update one network less often to stabilize the other), here realized as accumulation-window decoupling on a fixed on-policy buffer rather than as fresh-minibatch skipping.

**Caveat (fixed targets within an epoch).** The per-step advantages $\hat{A}_s$ and returns $G_s$ are computed once in PREPARE_FEEDBACK using the *old* critic and are frozen for the whole `optimize` epoch (Eq. (5)). Hence stepping the critic more often does *not* sharpen the current epoch's policy targets; its benefit is cross-iteration (a better baseline for the *next* epoch's GAE). A genuinely "moving" target would require recomputing returns more frequently within the epoch, which is out of scope here.

### 4.6 Periodic critic re-warmup (resample)

Interval decoupling (§4.5) cannot make the critic update *more* often than the policy: the finest cadence is one step per round. To let the critic periodically "catch up" to a moving policy we add a *re-warmup*, the same critic-only operation as the initial bootstrap. Every `critic_warmup_interval` optimizer rounds (checked after `optimize`; $\leq 0$ disables) the trainer runs WARMUPCRITIC with the periodic target `critic_rewarmup_steps`: it sets a fresh seed, SAMPLES new rollouts from the current policy, recomputes returns with the current critic via PREPARE_FEEDBACK, and fits *only* the critic, looping fresh resamples until it has taken $\sim$`critic_rewarmup_steps` optimizer steps. The initial bootstrap is the same routine with target `critic_warmup_steps`; the two targets are *independent* (a cold start usually wants more than a periodic catch-up). A burst never forwards or steps the policy and never advances `self.step` / `self.epoch`, so the policy still updates every epoch.

**Two regimes via `update_critic_in_optimize`.** With the default `True` (mode A) the critic also trains jointly inside `optimize`; bursts are an additional boost. With `False` (mode B) the critic is frozen inside `optimize` and trained *only* by the bootstrap and the periodic bursts – a policy/critic *alternation*: the policy advances every epoch on a frozen value baseline, and the critic re-fits in bursts (validated to require `critic_warmup_interval` $> 0$). Because the resample draws from the *current* policy, a burst aligns the critic to the present distribution – the "moving target" the per-epoch frozen returns (§4.5 caveat) cannot. The cost scales with `critic_rewarmup_steps` (each burst loops enough resampled rollouts to reach that many critic steps).

# 5 Theoretical Analysis

## 5.1 State-dependent baselines are unbiased and reduce variance

**Proposition 1** (Baseline)**.** *For any function $b(\boldsymbol{z}_s, t_s)$ of the state only,*

$$\mathbb{E}\Big[\textstyle\sum_s \nabla_\theta \log \pi_\theta(a_s \mid \boldsymbol{z}_s, t_s)\,(G_s - b(\boldsymbol{z}_s, t_s))\Big] = \nabla_\theta J(\theta),$$

*i.e. subtracting a state-dependent baseline leaves the policy gradient unbiased. The per-step gradient variance is minimized by $b \approx \mathbb{E}[G_s \mid \boldsymbol{z}_s, t_s] = V^\pi(\boldsymbol{z}_s, t_s)$.*

*Proof sketch.* For a fixed state, the baseline contributes zero in expectation:

$$\mathbb{E}_{a_s\sim\pi}\big[\nabla_\theta \log \pi_\theta(a_s \mid \cdot)\, b(\boldsymbol{z}_s, t_s)\big] = b(\boldsymbol{z}_s, t_s)\,\nabla_\theta \int \pi_\theta \,\mathrm{d}a_s = b(\boldsymbol{z}_s, t_s)\,\nabla_\theta 1 = 0.$$

Variance is a quadratic in $b$ minimized near the conditional mean of the return, which is the value function. □

Proposition 1 justifies replacing GRPO's group-mean baseline by the learned $V_\phi$: a state-conditioned baseline is the variance-optimal choice and is strictly more informative than a per-group constant.

## 5.2 GAE bias–variance

$A_s(\gamma, \lambda) = \sum_{l\geq 0}(\gamma\lambda)^l \delta_{s+l}$ interpolates between the one-step TD estimator ($\lambda$=0: low variance, biased by critic error) and the Monte-Carlo return minus baseline ($\lambda$=1: unbiased w.r.t. the true value, high variance). With a learned, imperfect $V_\phi$, an intermediate $\lambda$ (default 0.95) typically minimizes mean-squared advantage error. Because the reward is terminal, larger $\lambda$ propagates the single terminal signal further back along the trajectory, while $V_\phi$ shapes the intermediate credit.

## 5.3 Trust-region surrogate

Equation (5) is the PPO clipped surrogate: it lower-bounds the policy improvement while preventing destructive updates when $\rho_s$ drifts from 1. The on-policy ratio $\rho_s = 1$ at the first inner epoch (rollout policy = optimization policy), and clipping bounds the step as inner epochs reuse the rollout data.

## 5.4 Train–inference consistency

The actor's $\log \pi_{\text{old}}$ is produced by the *same* adapter forward and scheduler step used at training time, stored during rollout; the optimize-time forward recomputes $\log \pi_\theta$ identically up to the parameter update. This keeps $\rho_s$ an honest importance ratio—an invariant Flow-Factory enforces across all coupled algorithms.

# 6 Comparison with *Explicit Critic Guidance* (arXiv:2605.27736)

Liang et al. [1] propose a *state-aligned latent actor–critic* for diffusion post-training in which **the diffusion model serves as its own timestep-conditioned value function**, predicting values directly on noisy latent states. This enables trajectory-level PPO, stabilized by simple conditioning and *value pretraining*, and—because the value head shares the backbone—the learned critic can be reused for *inference-time steering*. They further train with multiple complementary rewards to mitigate reward hacking, and report gains over group-relative RL (GRPO) and actor–critic baselines on UNet- and DiT-based backbones.

Flow-Factory's PPO shares the same core thesis—*state-aligned value estimation on noisy latents with per-step PPO credit*—but makes the opposite engineering choice for the critic body. The paper's "diffusion-model-as-value-function" is exactly the **backbone+value-head critic** that Flow-Factory evaluated and deliberately *deferred* in favor of an independent, lightweight, model-agnostic critic. Figure 4 and Table 2 contrast the two.

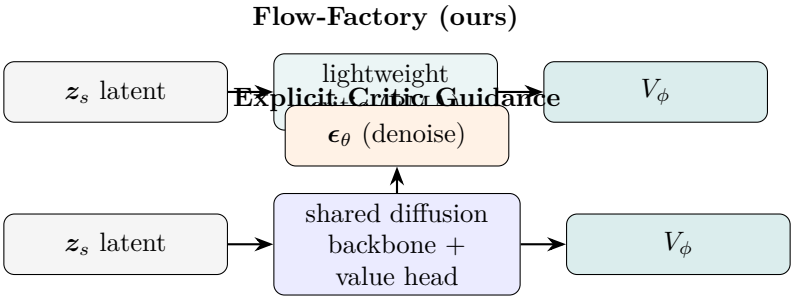


Figure 4: Critic body: independent lightweight network (ours) vs. the diffusion backbone reused as its own value function [1].

| **Aspect** | **Flow-Factory PPO (ours)** | **Explicit Critic Guidance [1]** |
|---|---|---|
| Critic body | independent lightweight net; attention-pool on latent tokens | diffusion backbone is its own value function (shared weights + value head) |
| State alignment | $V_\phi(\boldsymbol{z}_s, t_s)$ on noisy latent (state-aligned) | value predicted on noisy latent states (state-aligned) |
| Credit assignment | GAE per step | trajectory-level PPO, fine-grained credit |
| Value init | critic warmup (critic-only steps) | value pretraining strategies |
| Inference-time steering | not supported (follow-up) | critic reused for test-time steering (extra gains) |
| Multi-reward | terminal weighted sum; source-aware deferred | joint multi-reward to reduce reward hacking |
| Backbones | model-agnostic (conv / packed / video) | UNet and DiT |
| Cost | cheap: small critic, $O$(seq) pooling | heavier: a backbone forward per value query |

Table 2: Side-by-side comparison. Both are state-aligned actor–critic PPO for diffusion; the main axis of difference is the critic body and what it unlocks (cheap + model-agnostic vs. stronger value capacity + inference-time steering).

**Discussion.** The paper is strong evidence that the backbone-critic direction pays off, especially because a shared backbone (i) gives the value head the policy's rich representation, (ii) conditions on the full prompt/condition for free, and (iii) enables test-time steering. Flow-Factory's v1 trades some of that value-estimation capacity for a critic that is essentially free in memory/compute and works across every adapter without per-architecture surgery. The two designs are complementary: the lightweight critic is the safe default and reference baseline, while the backbone+value-head critic (and its inference-time steering and source-aware multi-reward) are the natural next upgrades, for which this PPO trainer already provides the GAE / dual-optimizer / checkpoint scaffolding.

# 7 Implementation Notes and Configuration

- **Files.** `trainers/ppo/{critic,gae,trainer}.py`, `hparams/training_args/ppo.py`, and `models/latent_geometry.py` (`compute_actual_latent_shape` / `latent_shape`). Example: `examples/ppo/lora/sd3_5/default.yaml`.
- **Paradigm.** Coupled paradigm: requires SDE dynamics (`Flow-SDE`, `Dance-SDE`, or `CPS`). Inherits `BaseTrainer`.
- **Distributed.** v1 targets DDP (the critic uses a second optimizer); DeepSpeed/FSDP support for the second optimizer is a follow-up. The dual-optimizer manual-accumulation path is verified for `bf16`; `fp16` (a shared `GradScaler` across two optimizers) is a follow-up.
- **Update-frequency decoupling.** `critic_update_interval` / `policy_update_interval` (in optimizer rounds) set per-network accumulation windows; see §4.5.
- **GAE assumption.** Configure a full, contiguous SDE trajectory (`scheduler.sde_steps: null`, `num_sde_steps = num_inference_steps - 1`) so the per-step bootstrap is exact.

| Hyperparameter | Default | Meaning |
|---|---|---|
| `critic_learning_rate` | 1e−4 | critic AdamW LR |
| `vf_coef` | 0.5 | value-loss weight $c_v$ |
| `gae_gamma` / `gae_lambda` | 1.0 / 0.95 | $\gamma$ / $\lambda$ |
| `value_clip_range` | 0.2 | value clip $\epsilon_v$ (None to disable) |
| `clip_range` | $\pm 10^{-4}$ | policy ratio clip $(c_-, c_+)$ |
| `normalize_advantage` | true | global advantage whitening |
| `critic_warmup_steps` | 30 | initial critic-only bootstrap target steps |
| `critic_warmup_interval` | 0 | periodic re-warmup every $N$ rounds ($\leq$0 off) |
| `critic_rewarmup_steps` | 0 | target critic steps per periodic re-warmup |
| `update_critic_in_optimize` | true | false $\Rightarrow$ critic only via warmup/bursts |
| `critic_update_interval` | 1 | step critic every $N$ rounds |
| `policy_update_interval` | 1 | step policy every $N$ rounds |
| `critic_hidden_dim` / `_num_layers` | 256 / 3 | critic width / depth |
| `critic_attn_heads` / `_num_query_tokens` | 8 / 1 | attention-pool config |
| `kl_beta` / `kl_type` | 0 / x-based | optional KL-to-reference |

Table 3: Key `PPOTrainingArguments` fields.

# 8 Limitations and Future Work

## 8.1 Follow-up: DiffAC-style perturbed-state critic training

**Idea.** Today both the *old values* (for GAE) and the in-loop critic regression target are evaluated on the *rollout* latents the policy actually visited (`all_latents` indexed per SDE step). A cheap, high-leverage upgrade—taken from DiffAC [2]—is to additionally (or alternatively) train the critic on *forward-perturbed* states: take the generated clean latent $\boldsymbol{z}_S$ and re-noise it to many noise levels $t$ in closed form,

$$\boldsymbol{z}_t = (1 - \sigma_t)\, \boldsymbol{z}_S + \sigma_t\, \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{I}), \;\; \sigma_t = \texttt{flow\_match\_sigma}(t), \tag{9}$$

and regress the critic on each perturbed state toward that sample's terminal reward, $V_\phi(\boldsymbol{z}_t, t) \to R(\boldsymbol{z}_S)$ (DiffAC Eq. 8/9). One rollout then yields an *arbitrarily large*, space-covering set of $(\boldsymbol{z}_t, t, R)$ training points instead of the $S$ sparse on-trajectory states, which lowers value-estimation

variance and improves time coverage. Under the consistency condition (DiffAC Lemma 4.2: when the backward SDE matches the forward perturbation process, $\boldsymbol{z}_t \sim p_{t0}(\cdot \mid \boldsymbol{z}_S)$ is distributed as the true rollout marginal $q_t$) these perturbed states are a valid, on-distribution proxy for the states the GAE consumes.

**Why it is cheap here.** The forward-noising of Eq. (9) is *already implemented* in the AWM trainer (`trainers/awm.py`: `noised_latents = (1 - sigma) * clean_latents + sigma * noise`), and the critic forward already accepts an arbitrary (latents, $t$). The change is therefore self-contained.

**Plan.**

1. **New hparam** `critic_target: {gae, perturb_terminal}` (default `gae`, preserving current behavior), plus `critic_perturb_samples` (number of noise levels drawn per clean latent) and a $t$-sampling strategy (reuse AWM's `time_sampling_strategy`).
2. **Critic training path.** In `perturb_terminal` mode, replace the rollout-state critic batches in `_fit_critic_only` (and the in-`optimize` critic branch) with perturbed states from Eq. (9), regressed to the per-sample terminal reward $R(\boldsymbol{z}_S)$ with a *plain* MSE (no cross-step bootstrap; this is the $\gamma{=}\lambda{=}1$ Monte-Carlo target). `_critic_value_loss` is reused; `value_clip_range` may be set `None` to match DiffAC's plain MSE.
3. **Actor untouched.** The policy still uses PPO-clip; GAE still runs over the rollout states, but now bootstraps off a better-calibrated $V_\phi$. (This is the variance-reduction half of DiffAC; the full method also moves the *actor* to a one-step REINFORCE/DDPG on perturbed states, which is out of scope for this follow-up.)

**Caveats to respect.**

- **Monte-Carlo target only.** Perturbed states are mutually independent (no "next" state), so the critic must regress directly to $R(\boldsymbol{z}_S)$; the per-step bootstrap of `compute_gae` does not apply to these points.
- **Timestep scale.** The $t$ used to noise (Eq. (9)) and the $t$ fed to the critic's time embedding must be on the same scheduler scale, or the critic conditions on the wrong noise level.
- **Consistency gap.** The proxy is exact only when the policy stays consistent with the forward process; as RL pushes the policy off-distribution the gap grows (DiffAC restores it with a score-matching–refit reference; we only borrow the critic-side benefit, so treat this as an approximation).
- **Not full DiffAC.** Training only the critic on perturbed states is an incremental strengthening of our PPO baseline, not a reproduction of DiffAC.

### 8.2 Other future work

- **Backbone+value-head critic** (as in [1]): stronger value capacity and free conditioning, at ∼2× memory/compute; needs per-architecture feature taps. Evaluated and deferred.
- **Inference-time steering** with the learned critic (requires the shared backbone).
- **Text / condition conditioning** of the critic (pooled or full prompt embeddings) and **source-aware multi-reward** aggregation.
- **DeepSpeed/FSDP** support for the critic's second optimizer.

## References


[1] Zhengyang Liang, Qihang Zhang, Ceyuan Yang. Explicit Critic Guidance for Aligning Diffusion Models. arXiv:2605.27736, 2026. https://arxiv.org/abs/2605.27736

[2] Xiangxin Zhou, Liang Wang, Yichi Zhou. Stabilizing Policy Gradients for Stochastic Differential Equations via Consistency with Perturbation Process (DiffAC). ICML 2024; arXiv:2403.04154. https://arxiv.org/abs/2403.04154